

MPC using Shamir Secret Sharing: BGW

2025

1 Introduction

We mentioned before in this course that MPC can be realised via either garbled circuits or secret sharing. Both approaches have different advantages and disadvantages in terms of computation and communication cost.

We defined what secret sharing is before and looked at different secret sharing schemes. Shamir's (t, n) threshold secret sharing scheme is rather versatile and it therefore underpins real world implementations. We now look at the “origin” of modern MPC protocols: the BGW protocol.

1.1 MPC via Secret Sharing

The idea behind MPC using Shamir secret sharing is that we express the function that the multiple parties wish to compute as a “circuit” with wires that represent values over $\mathbb{Z}_p$. The secret input wires are shared out among all parties using Shamir's scheme, and then the parties interact (depending on the circuit) to jointly compute intermediate values and the final output.

Note. Any function that can be efficiently computed can be represented as a Boolean circuit; that is a sequence of **And** and **Or** (or **xor**) operations. There are various ways in which efficient circuits for some given input function can be found (hardware design tools specialise on this). We can represent any such Boolean circuit as a function over $\mathbb{Z}_p$ (with $p > 2$) by encoding its values as 0 and 1 in $\mathbb{Z}_p$.

Caution. Bear in mind, especially when reading/using different resources, that in my notation, the threshold t equates to the number of parties that must come together to reconstruct. Therefore, it equates to the number of coefficients in the secret polynomial(s) for sharing. This means that a threshold t implies working with polynomials of degree $t - 1$. Some resources though interpret threshold as though you need to cross that

threshold in order to reconstruct: so a threshold t equates to a degree t polynomial, and therefore you need $t + 1$ parties to reconstruct. This can be rather confusing !

2 Computing shared addition and multiplication

Because we will represent any function as a circuit with two basic gates, addition of two values and multiplication of two values, we need to understand how to compute these gates when their inputs are shared.

2.1 Addition

We want to compute $c = a + b$ ($a, b, c \in \mathbb{Z}_p$), whereby a and b are shared via Shamir's scheme, and the output c should be shared too. Recall that we share values by defining a secret random polynomial that evaluates at 0 to the given value:

$$\begin{aligned} f(x) &= a + f_1x + f_2x^2 + \dots f_tx^t \\ g(x) &= b + g_1x + g_2x^2 + \dots g_tx^t \end{aligned}$$

Clearly, the polynomial corresponding to $c = a + b$ is given by simply adding the polynomials $f(x) + g(x)$ component wise to get $h(x) = a + b + h_1x + h_2x + \dots h_tx^t$, whereby $h(0) = a + b$. Notice that because $f(i) + g(i) = (f + g)(i)$, each party “gets” $h(i)$ by simply adding their shares.

Example. Let us consider a scheme with three players, and all players must come together to recover the secret value; i.e. we have $(3, 3)$ scheme. There are two secret values that are shared among the players: $a = 2, b = 3$. The players are asked at some point to derive the sum of a and b without reconstructing a and b . They can do this via simply adding up their shares of a and b . In this example we assume that all operations are done in $\mathbb{Z}_11$, and the table below shows the situation from the perspective of the players (each player can only see the values in their column).

Value	P_1	P_2	P_3
$a = 2$	9	2	3
$b = 3$	6	0	7
$a + b = 5$	4	2	10

The secret sum can be reconstructed from the shares by finding the unique polynomial $h(x)$ that goes through the three points $(1, 4)$, $(2, 2)$, and $(3, 10)$. You can use Lagrange's method to get there, and you should find that $h(x) = 5 + 5x + 5x^2$. The secret polynomials to share out a, b were in fact: $f(x) = 2 + 3x + 4x^2$ and $g(x) = 3 + 2x + x^2$.

2.2 Multiplication by a public constant value

Multiplication by a publicly known constant value can be realised in a simple way: because the value is a constant and public knowledge, it does not need to be secret shared during the computing. Therefore, all parties are assumed to have it, and all parties simply multiply their share with it. This works because

$$c \cdot f(x) = c \cdot a_0 + c \cdot a_1 x^1 + \dots$$

Note that all the a_i were randomly chosen and the shares are (as a consequence) also random numbers, $c \cdot f(i)$ is still random (c is fixed thus multiplication with it is a one-to-one mapping from $\mathbb{Z}_p^*$ to $\mathbb{Z}_p^*$).

2.3 Multiplication

Dealing with a multiplication (at a first glance) no different to dealing with an addition. We want to compute $c = a \cdot b$ ($a, b, c \in \mathbb{Z}_p$), whereby a and b are shared via Shamir's scheme, and the output c should be shared too. Recall that we share values by defining a secret random polynomial that evaluates at 0 to the given value:

$$\begin{aligned} f(x) &= a + f_1 x + f_2 x^2 + \dots f_t x^t \\ g(x) &= b + g_1 x + g_2 x^2 + \dots g_t x^t \end{aligned}$$

Clearly, the representation $c = a \cdot b$ is given by the product of the polynomials $f(x) \cdot g(x)$, thus $h(x) = a \cdot b + h_1 x + h_2 x^2 + \dots h_t x^{2t}$, whereby $h(0) = a \cdot b$. Notice that because $f(i) \cdot g(i) = (f \cdot g)(i)$, each party “gets” $h(i)$ by simply multiplying their shares.

However, because the resulting polynomial $h(x)$ now has degree $2t$ now need $2t$ players rather than t to recover the secret. Therefore if $t \geq n/2$, then reconstruction is in fact no longer possible. Another issue is that the multiplication of two shares by the players lead to the situation where the resulting polynomial/shares are no longer uniformly distributed¹

Therefore we must employ a “trick” that requires interaction between all parties to perform what is called a “degree reduction” after each multiplication. This degree reduction essentially consist of sampling a new sharing of the product, and this can be done without revealing the product!

It works as follows. Each party takes their share $h(i)$ and produces a new (t, n) sharing for it. This means, that they produce a degree t polynomial with $h(i)$ as the constant

¹As an intuition observe that the choice of f and g by randomly sampling coefficients, in principle, allows that they could be “irreducible” polynomials. However the product $h = f \cdot g$ is definitely not irreducible, so it cannot be a random polynomial.

term, and randomly sampled coefficients otherwise. They then give shares to all other parties. This is where the (costly) interaction comes in. Once all parties have received all shares, each party reconstructs a new sharing for their share of the secret product.

In slightly more mathematical terms, after computing the individual products, we have that the overall product is given by $h(0) = (f \cdot g)(0) = a \cdot b = c$; and party P_i has $h(i)$. Remember that the Lagrange reconstruction for $h(x)$ is $h(x) = \sum_{i=1}^n \delta_i(x)h(i)$. Therefore, we can know that $h(0) = \sum_{i=1}^n \delta_i(0)h(i)$. The terms $\delta_i(0)$ are terms from the public recombination vector, and using $r_i = \delta_i(0)$ we can express $c = h(0) = \sum_{i=1}^n r_i \cdot h(i)$. The degree reduction trick, as explained in the previous paragraph, then does the following:

Degree reduction:

- Party P_i samples $\delta'_i(x)$ with degree $\leq t$ such that $\delta'_i(0) = h(i)$, and shares $\delta'_i(j)$ with party P_j .
- Party P_j computes $H(j) = \sum_{i=1}^n r_i \delta'_i(j)$.

Observe that $H(0) = \sum_{i=1}^n r_i \delta'_i(0) = \sum_{i=1}^n r_i h(i) = c$. Therefore the points $H(i)$ are points on a polynomial H of degree at most t that evaluates to c on 0. This trick works because, per construction, we ensure that all polynomials produce $h(i)$ when evaluated at $x = 0$, and thus their linear combination using the public recombination vector retains the product c . This may look confusing, which is why we now look at a concrete example.

Example. We consider again a situation with three players, and threshold $t = 2$. The secret values $a = 5$ and $b = 2$ are shared out among all players using some secret degree 1 polynomials over $\mathbb{Z}_{11}$. They use their shares to compute shares of $a \cdot b$, and then come together to reconstruct the product.

Value	P_1	P_2	P_3
$a = 5$	2	10	7
$b = 2$	0	9	7
$a \cdot b = 10$	0	2	5

You can pretend to be the players who reconstruct the final degree 2 polynomial from the shares, and evaluate it at 0 to find that the constant term is indeed 10. You could also use the public recombination vector. Notice that because we initially set the threshold to $t = 2$, we work with polynomials with degree 1.

Continuing with the example: the reconstruction formula for the product is $y_1 \cdot 3 + y_2 \cdot 8 + y_3 \cdot 1 \pmod{11}$. It is based on finding $\delta_1(0) = 3$, $\delta_2(0) = 8$, and $\delta_3(0) = 1$ (for a degree 2 polynomial). Plugging the shares $(y_1, y_2, y_3) = (0, 2, 5)$ into this formula, we also find that the product is 10.

However, we do not want to work with the shares 0, 2, 5 of the secret product because we know that these shares are not uniformly distributed. Therefore we are now going to follow the degree reduction process, which also produces a new (t, n) sharing of our product.

The degree reduction works by creating new (t, n) sharings for the shares of the product that the parties P_i hold. In this example the players randomly select the polynomials:

$$P_1 : 0 + 3x$$

$$P_2 : 2 + 6x$$

$$P_3 : 5 + 2x.$$

With these polynomials, P_1 produces shares for 0 as 3, 6, 9, P_2 produces shares for 2 as 8, 3, 9, and P_3 produces shares for 5 as 7, 9, 0. They then interact to distribute the shares, which means that after distribution, the situation looks like:

Player				
P_1	3	8	7	$c_1 = 3$
P_2	6	3	9	$c_2 = 7$
P_3	9	9	0	$c_3 = 0$

Each player can use their shares to reconstruct a new share for the actual product using the public recombination formula (remember with y_i being the shares that the players hold). Thus after degree reduction the players hold the shares of the secret product (10) as (3, 7, 0). You can check that also this sharing given 10 with the public recombination formula.

Furthermore you can check that the new shares are indeed a valid (t, n) sharing by reconstructing the secret polynomial. You should find that it is $10 + 4x$.

3 BGW Protocol for Secure Circuit Evaluation

We are now ready to give a full protocol for the secure evaluation of any arithmetic circuit, which goes back to the researchers Ben-Or, Goldwasser, and Wigderson. The protocol

provides perfect security against passive adversaries: recall this implies that all users behave honestly, i.e. they follow the protocol (even when taken over by an adversary). At most t out of n users may be corrupted.

Any arithmetic circuit consists of gates. In particular, we assume that the function that we wish to jointly compute is given as a sequence of addition, multiplication by constant, and multiplication gates. The sequence has a clear ordering, all parties know this ordering (i.e. they all compute the same function, in the same way). The evaluation of the circuit then proceeds in three steps.

Input Sharing. Each party P_i with input x_i produces an (n, t) sharing of their input. They then distribute shares to the other parties.

Computation. Parties now process the circuit gate by gate, whereby they maintain the fact that if the inputs to a gate are (t, n) secret shared, then the outputs to the gate are also (t, n) secret shared. This requires no overhead for addition and multiplication by constant gates, but whenever a multiplication is required, then communication between all parties is required.

Output reconstruction. When all gates have been processed, the parties use Lagrange interpolation (i.e. the public reconstruction vector) to reconstruct the output. Therefore each party sends their share to all other parties (securely).

The security of this protocol follows immediately from the security of the Shamir secret sharing scheme, and the fact that we have ways to compute the gates in a way that maintains the property of (t, n) secret sharing.

Slightly more formal. Reasoning about the security of the BGW protocol in the real/ideal setting requires us to be a little bit more precise about how the properties of Shamir secret sharing help us to build a simulator so that an adversary cannot tell if they are interacting with a real or an ideal version of the protocol.

Firstly notice that all values that are exchanged during the circuit evaluation, i.e. the initial sharings of the inputs, and any potential shares during the evaluation of multiplication gates, are distributed uniformly at random. Thus any messages during the execution in the real world can be replaced by random values and this cannot be distinguished at all by any observer.

The perhaps most interesting aspect is in the final output reconstruction step. The output y of the evaluation is public, thus the corrupt parties know it, and the simulator has access to it as well. During reconstruction, any corrupt party gets the final output

shares from the all other parties including the honest parties. The question is if the shares from the honest parties give some additional information to the corrupt parties or not. Equivalently, whether these shares can be produced by the simulator without access to secret inputs of the honest party, or not.

Notice that the t corrupt parties have t shares already. Given t shares plus $y = f(0)$, the corrupt parties can determine f (the polynomial that gives the output sharing) and therefore they can actually determine the shares of the honest parties. The same behaviour can be implemented in the simulator: it uses the t corrupted shares plus y in order to simulate the output shares of the honest parties. The simulator can do this without knowing anything about their inputs, thus no information about their inputs can be leaked to the corrupt parties.